

CS 550 Operating Systems

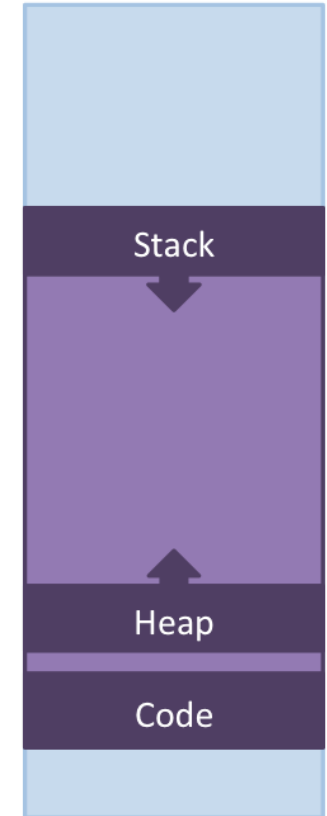
Spring 2026

Segmentation

Internal fragmentation with the dynamic relocation approach

- The dynamic relocation approach puts the entire address space of each process in memory.
 - a big chunk of “free” space right in the middle, between the stack and the heap - **internal fragmentation**.
 - More generally, internal fragmentation occurs when the memory allocator leaves extra space empty inside of a block of memory that has been allocated for a client.
- How to solve the problem still using the “base and bound” idea?

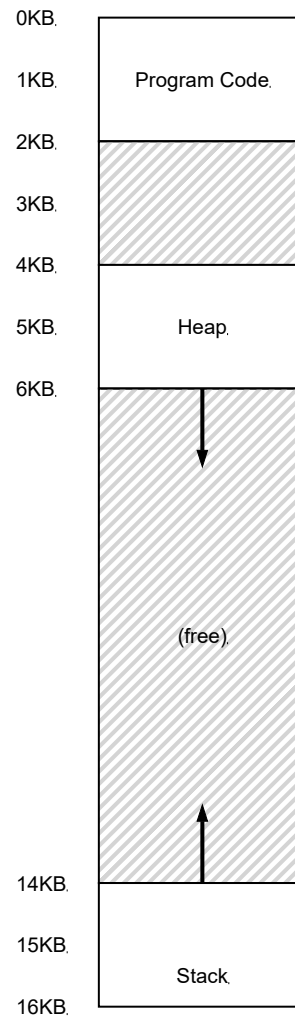
Physical Memory

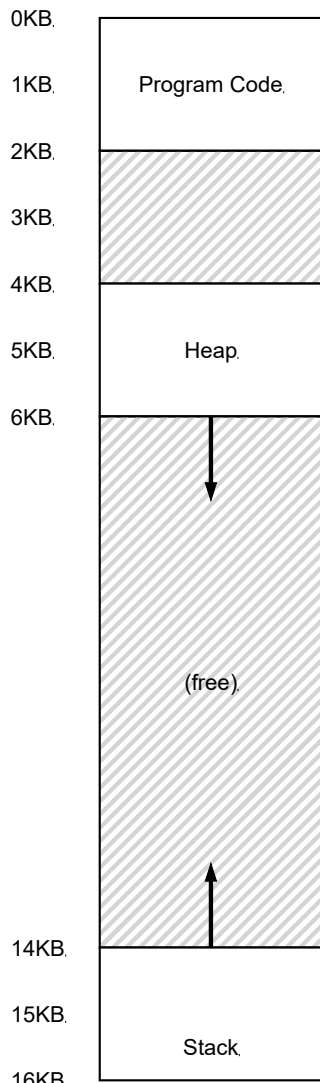


Segmentation

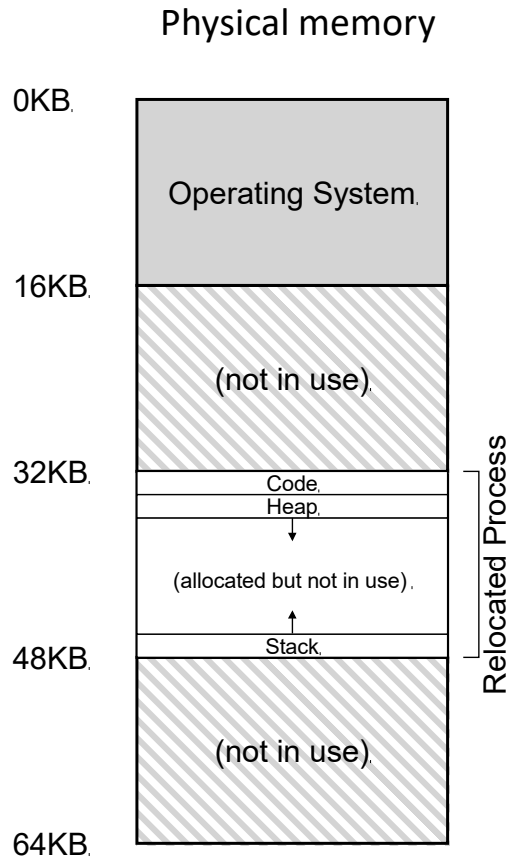
- Just one pair base/bound registers → a pair of base/bound registers for each **logical segment** of the address space
 - A **segment** is a contiguous portion of the address space of a particular length.
 - Segmentation allows OS to place segments in different parts of physical memory.
 - Three typical logic segments in a address space: code, stack, and heap

Process address space

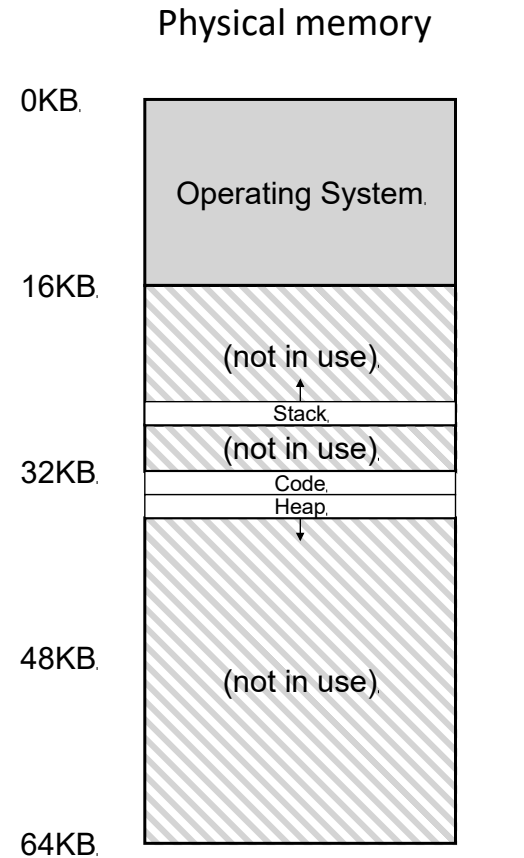




Process address space



Dynamic relocation
(single base/bound pair)

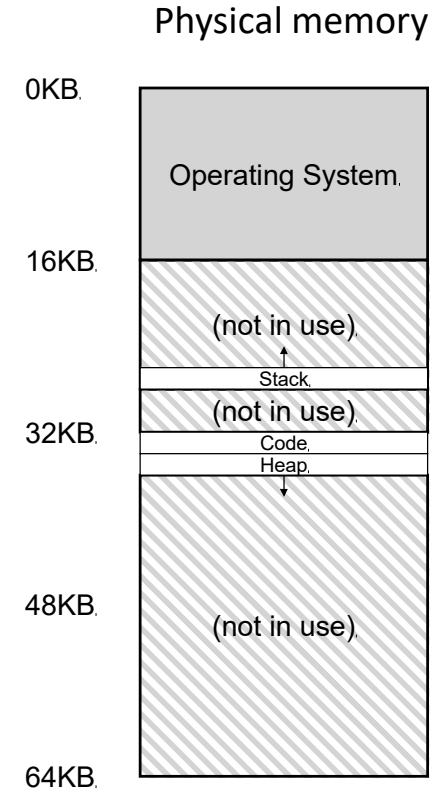


Segmentation
(different base/bound pairs
for different logic segments)

Segmentation hardware support

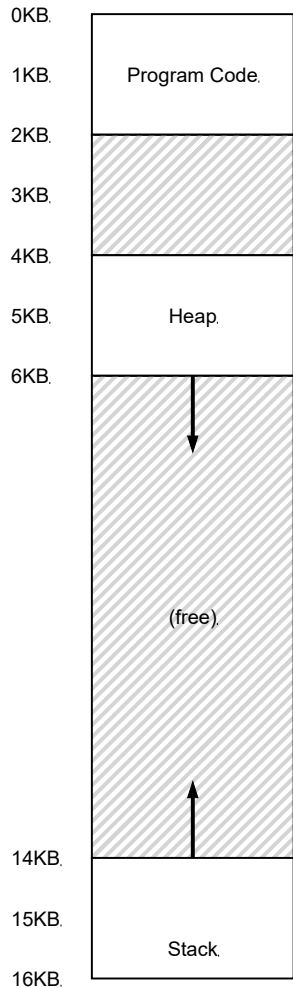
- Compared to the dynamic relocation approach, what change we need to make for the MMU?
 - Different pairs of base/bound registers in MMU for the different logic segments

Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K



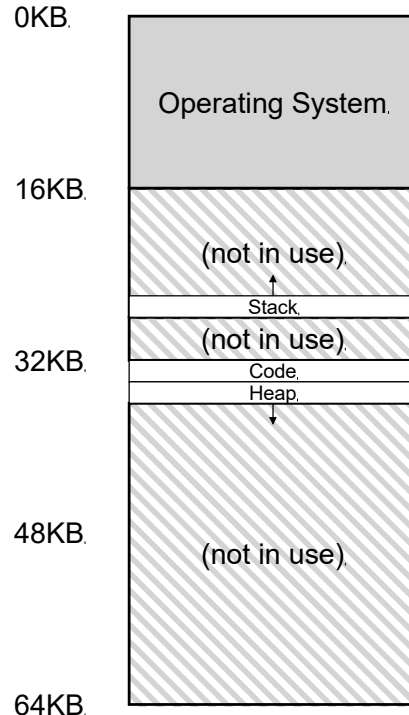
Segmentation address translation example

Process address space



Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

Physical memory



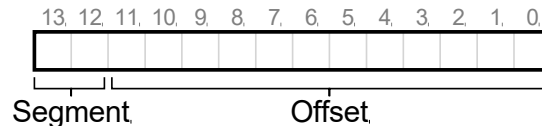
- Access VA 100
 - Code segment
 - $PA = Base + VA = 32K + 100 = 32868$
- Access VA 4200
 - Heap segment
 - $PA = Base + VA = 34K + 4200 = 39016?$ (crossed out with a red X)
 - $PA = BASE + VA - SEG_START = 34K + 4200 - 4K = 34920$
- Access VA 7000
 - illegal address, Segmentation fault

Which segment we're referring to?

- Given a VA, how can the OS determine which segment it is referring to?
 - Explicit approach
 - Implicit approach

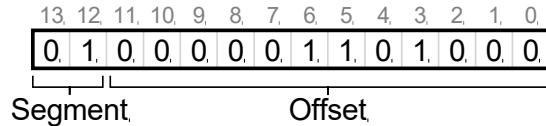
The explicit approach

- Chop up the address space into segments based the top few bits of the VA.
 - Suppose in a 14-bit system (i.e., a memory address has 14 bits), there are three segments: code, heap, and stack.
 - What is the size of an address space?
 - Then how many bits are needed to index these segments?
 - What's the maximum size of each segment?



The explicit approach

- For the previous example, top two bits: 00 – code; 01 – heap; 11 – stack.
 - Example: accessing VA 4200 (decimal)



```
// get top 2 bits of 14-bit VA
Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
// now get offset
Offset = VirtualAddress & OFFSET_MASK
if (Offset >= Bounds[Segment])
    RaiseException(PROTECTION_FAULT)
else
    PhysAddr = Base[Segment] + Offset
    Register = AccessMemory(PhysAddr)
```

SEG_MASK?

SEG_SHIFT?

OFFSET_MASK?

The implicit approach

- Implicit approach: based on how address is formed
 - from CPU (program counter) → code
 - based on stack pointer → stack
 - else → heap

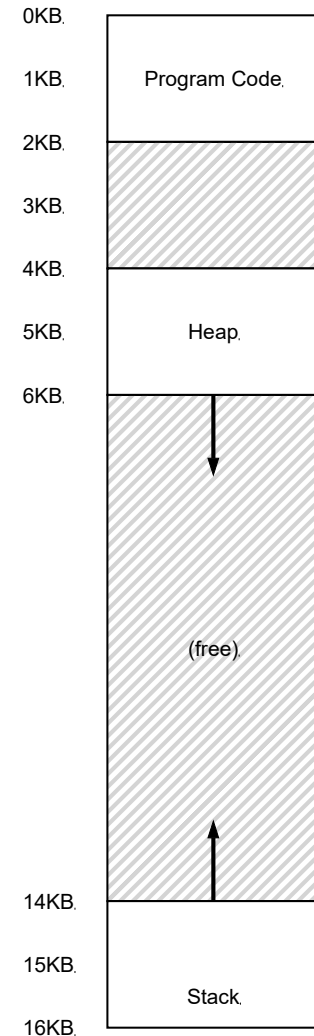
Other HW supports

- Stack segment: grows backwards
 - The HW needs to know which way the segment grow

Segment	Base	Size	Grows Positive?
Code	32K	2K	1
Heap	34K	2K	1
Stack	28K	2K	0

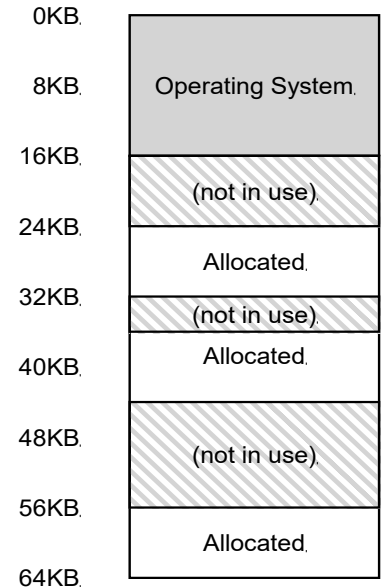
- Support for sharing: to save memory, sometimes it is useful to share certain memory segments between address spaces (e.g., code). What we need to do?

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write



OS Support

- How should OS do context switching?
 - The segment registers are saved and restored when switching between different address spaces.
- Managing free space in physical memory
 - Easy with the dynamic relocation approach when each address space is of the same size.
 - With the segmentation approach, each segment might be a different size. Gradually, free space is separated into small blocks and is interspersed by allocated memory → *external fragmentation*.
 - How can we solve this problem?



– Solution: memory compaction

- Compact physical memory by rearranging the existing segments
- What is the downside of this approach?
 - Expensive: OS needs to stop the currently running process, move its segments in physical memory to make larger continuous space, update the segment registers.

– Simpler solution: free-list management algorithm.

